

Live iPhone Debugging

This guide records the command-line workflow used to debug Bluefruit Playground on a physical iPhone. It is useful for Bluetooth connection, notification, and sensor-update problems that are difficult to diagnose from Xcode's console.

Prerequisites

- The iPhone is connected by USB, unlocked, and trusted by the Mac.
- Xcode supports the iOS version installed on the phone.
- The app has been built with the Debug configuration and is installed on the phone.
- Commands are run outside a restricted sandbox when device access is required.

1. Find the device

```
xcrun devicectl list devices
```

Use the physical iPhone identifier from the **Identifier** column. For example:

```
1058A148-D134-5843-8504-2B004DA3EE73
```

If the phone is missing, confirm the USB cable, unlock it, and accept its trust prompt. Xcode may see a device when a sandboxed command cannot; run the command with the required local-device permission in that situation.

2. Confirm developer support is available

```
xcrun devicectl device info processes --device <device-id>
```

The first request mounts Xcode's Developer Disk Image (DDI) on the phone. If it reports that the device is locked, unlock it and retry. If Cryptex/DDI mounting fails, reboot the phone, reconnect by USB, unlock it, and retry before looking at application code.

3. Find the app bundle identifier

```
xcrun devicectl device info apps --device <device-id>
```

For this project the installed application is:

```
com.michaelzfreeman.BluefruitPlayground
```

4. Launch the installed app

```
xcrun devicectl device process launch \  
  --device <device-id> \  
  com.michaelzfreeman.BluefruitPlayground
```

This launches the build already installed on the phone. To see the current process identifier (PID), run:

```
xcrun devicectl device info processes \  
  --device <device-id> \  
  --json-output /private/tmp/bluefruit-processes.json
```

Look for the **Bluefruit Playground** executable in the output. In the current session it was PID **443**.

5. Attach LLDB

Start the debugger:

```
xcrun lldb
```

At the **(lldb)** prompt, select the iPhone and attach to the PID:

```
device select <device-id>  
device process attach -p <pid>  
continue
```

device select chooses the physical-device debug service. **device process attach** pauses the already-running app. **continue** resumes it so the user can exercise the app while LLDB remains connected.

If **continue** initially says the process must be launched, wait for the attach to finish: LLDB may report the stop asynchronously. Once it shows the target as **Bluefruit Playground**, run **continue** again.

6. Add source breakpoints

A source breakpoint is temporary: it changes neither the source code nor the phone. LLDB maps a file and line number to the compiled program and pauses when execution reaches it.

```
breakpoint set -f BleManager.swift -l 426
breakpoint set -f BleManager.swift -l 393
breakpoint set -f AdafruitBoardsManager.swift -l 116
breakpoint set -f BlePeripheral.swift -l 483
breakpoint set -f BlePeripheral.swift -l 583
breakpoint set -f BlePeripheral.swift -l 587
```

These locations mean:

Breakpoint	Code path	What it proves
<code>BleManager.swift:426</code>	<code>didDisconnectPeripheral</code>	CoreBluetooth dropped the link; inspect its error and the call stack.
<code>BleManager.swift:393</code>	<code>didConnect</code>	The peripheral connected.
<code>AdafruitBoardsManager.swift:116</code>	reconnect restoration	The app is re-discovering services and restoring sensor setup.
<code>BlePeripheral.swift:483</code>	<code>setNotify</code>	The app asked CoreBluetooth to subscribe to a sensor characteristic.
<code>BlePeripheral.swift:583</code>	notification-state callback	The board accepted or rejected that subscription; inspect <code>error</code> .
<code>BlePeripheral.swift:587</code>	characteristic-value callback	A sensor packet reached the app; this should recur while live readings are active.

The notification subscription call is:

```
private func setNotify(with command: BleCommand) {
    let characteristic = command.parameters![0] as! CBCharacteristic
    let enabled = command.parameters![1] as! Bool
    let identifier = handlerIdentifier(from: characteristic)

    if enabled {
        let handler = command.parameters![2] as? ((Error?) -> Void)
        notifyHandlers[identifier] = handler
    } else {
        notifyHandlers.removeValue(forKey: identifier)
    }
}
```

```
}  
  
    peripheral.setNotifyValue(enabled, for: characteristic)  
}
```

When LLDB stops, inspect the current source with `frame source`, inspect local variables with `frame variable`, and resume with `continue`.

7. Interpret a frozen sensor screen

Use the breakpoint sequence as a decision tree:

```
setNotify -> notification-state callback -> value callback repeats
```

- No `setNotify`: opening the screen did not request sensor updates. Inspect the view controller and service setup that should enable the sensor.
- `setNotify`, then callback with an error: the characteristic does not support notifications or the board rejected the request.
- Successful notification-state callback, but no value callback: the board is subscribed but not sending samples. Check the service UUID, measurement period, and characteristic configuration.
- Repeated value callbacks, but a frozen screen: BLE is healthy; inspect decoding and UI/main-thread updates.
- Disconnect callback: inspect the CoreBluetooth error, then verify that the reconnect path reaches `AdafruitBoardsManager.didReconnectToPeripheral` and restores the board services.

Current observation

With the Accelerometer screen open, LLDB received no value callbacks for 30 seconds. After adding breakpoints for `setNotify` and notification-state changes, reopening the screen did not reach either breakpoint. The immediate follow-up is therefore to trace the screen controller's attempt to enable the accelerometer, rather than to investigate model rendering or sensor-value decoding.

End the session

At the LLDB prompt, detach without terminating the app:

```
process detach  
quit
```

Or terminate the app deliberately:

```
xcrun devicectl device process terminate \  
  --device <device-id> \  
  --pid <pid>
```